Algorithm	Time Complexity	Explanation of Time Complexity
BFS (Breadth-First Search)	O(V + E)	Each vertex (V) is visited once, and each edge (E) is explored at most once in an adjacency list representation.
DFS (Depth-First Search)	O(V + E)	Like BFS, it traverses all vertices and edges once. The recursion/stack operations do not change asymptotic cost.
SCC (Strongly Connected Components, e.g., Kosaraju/Tarjan)	O(V + E)	Runs DFS multiple times (Kosaraju: 2 DFS + graph transpose; Tarjan: single DFS with low-link values). Still linear in graph size.
Topological Sort	O(V + E)	Based on DFS or in-degree counting (Kahn's algorithm). Each node and edge processed once.
Kruskal's Algorithm (MST)	O(E log E)	Sorting all edges dominates. Union-Find operations are nearly constant time (amortized α(V)).
Dijkstra's Algorithm	O((V + E) log V) with priority queue	Extract-min operations on a heap take log V. Each edge is relaxed once, leading to E log V cost in total.

Huffman Encoding	O(n log n)	Building priority queue (min-heap) for n symbols takes O(n), but combining nodes requires log n operations repeatedly.
LCS (Longest Common Subsequence, DP)	O(m × n)	A 2D DP table of size `m × n` is filled, where `m` and `n` are lengths of the two sequences. Each cell computed in O(1).
0/1 Knapsack (DP)	O(n × W)	DP table of size `n × W`, where `n` = number of items, `W` = capacity. Each entry computed in O(1).

MST (Maximum-cost Spanning Tree) Keywords

Look for these keywords or patterns:

- **Spanning tree**
- **Minimum cost to connect all nodes**
- **Minimum total weight to connect all vertices**
- **Connect all cities/towns with minimum cost**
- **No cycles** or **tree structure**
- **Kruskal's algorithm**
- **Network design**, **wiring**, **cables**, **pipes**, **roads**

🔍 Example phrases:

- "What is the minimum cost to connect all computers in a network?"
- "Find the minimum spanning tree of the graph."

🧭 SSSP (Single Source Shortest Path) Keywords

Look for:

- **Shortest path from source to target**
- **Minimum distance from one node to all others**
- **Shortest time**, **least cost**, **quickest route**
- **Path from a given source**
- **Dijkstra's algorithm**
- **Source and destination** explicitly mentioned

🔍 Example phrases:

- "Find the shortest path from node A to node B."
- "What is the minimum time to travel from city X to all other cities?"

SCC – Strongly Connected Components

Definition:

In a **directed graph**, an **SCC** is a **maximal subset of vertices** such that **every vertex is reachable from every other vertex** in the subset (via directed paths).

Key Points:

- **Applies to directed graphs only**.
- A graph can have multiple SCCs.

- Algorithms: **Kosaraju's**
- Often used in:
    - Detecting cycles in directed graphs.
    - Analyzing structure in dependency graphs (like compilers, networks).

🔍 **Example:**

mathematica
CopyEdit

```
A → B → C → A
D → E → F → D
```

- There are **two SCCs** here: {A, B, C} and {D, E, F}.

🎨 Bipartite Graph

Definition:

A graph (usually **undirected**) is **bipartite** if its **vertices can be divided into two disjoint sets** such that **every edge connects a vertex in one set to a vertex in the other**, and there are **no edges between vertices of the same set**.

Key Points:

- Applies to **undirected graphs** (though some algorithms apply to directed versions).
- No **odd-length cycles** allowed.
- Can be tested via BFS or DFS.
- Applications:
    - **Matching problems**, job assignments, network flow.
    - Modeling two-group systems like people and jobs, users and items.

🔍 **Example:**

mathematica
CopyEdit

```
Set U = {A, C}, Set V = {B, D}
Edges: A–B, C–D
```

This is a bipartite graph.

🗒️ Summary Table

Feature	SCC (Strongly Connected Component)	Bipartite Graph
Graph type	**Directed** only	Usually **Undirected**
Definition	Maximal set where each vertex can reach every other via directed paths	Vertices can be split into two sets with no intra-set edges
Cycle allowed?	✅ Cycles exist within SCCs	❌ Odd-length cycles are **not** allowed
Use cases	Web crawling, compilers, dependency graphs	Matching, scheduling, recommendation systems
Detection algorithms	Kosaraju	BFS (for coloring), DFS
Example	A → B → C → A	A—B, C—D (no A—C or B—D)

🎯 Real-World Analogy

- **SCC**: Think of a **closed group** of websites where each site links to every other — you can surf from any page to any other within the group.
- **Bipartite Graph**: Think of **job assignments** — jobs are one group, workers another; each edge represents a person qualified for a job.

🔹 Greedy Algorithm (Basics)

Definition: A greedy algorithm builds up a solution piece by piece, always choosing the option that looks best at the moment (locally optimal choice).

Key idea: It doesn't reconsider choices once made.

When it works: If the problem has the greedy choice property (a global optimum can be reached by choosing local optima) and optimal substructure (an optimal solution can be built from optimal solutions of subproblems).

✅ Examples of problems solved by greedy:

1. Activity Selection – Select max number of non-overlapping activities.

2. Fractional Knapsack – Take items in order of value/weight until full.

3. Minimum Spanning Tree – Kruskal's or Prim's algorithm.

4. Shortest Path in Graph – Dijkstra's algorithm.

🔹 Huffman Encoding (Basics)

What it is: A compression algorithm (lossless) that assigns variable-length binary codes to characters, shorter codes to more frequent characters.

Type: Greedy algorithm.

Steps:

1. Count the frequency of each character.

2. Insert all characters into a min-priority queue (based on frequency).

3. While more than one node remains:

Remove two nodes with the lowest frequency.

Create a new node with frequency = sum of the two.

Make the two nodes children of the new node.

Insert the new node back into the queue.

4. The final node is the root of the Huffman Tree.

5. Assign codes:

Go left → 0,

Go right → 1.

Example:

Characters: {a: 5, b: 9, c: 12, d: 13, e: 16, f: 45}

Huffman encoding might generate codes like:

f = 0

c = 100, d = 101

a = 1100, b = 1101

e = 111

So the most frequent character (f) gets the shortest code.

👉 In short: Greedy = take the best immediate option. Huffman = a greedy way to compress data.

🧺 0/1 Knapsack

✅ Keywords / Clues:

- **"Either take it or leave it"**
- **"Cannot break items"**
- **"Item must be taken as a whole"**
- **"Pick at most one of each item"**
- **"Binary choice**: include or exclude"
- **"Only 1 copy** per item"
- Items are **indivisible**
- Usually implies **discrete choices**
- Often stated as **“select a subset of items”**

🧠 Think: All or nothing

🍰 Fractional Knapsack

✅ Keywords / Clues:

- **"Can take fractions of items"**
- **"Items are divisible"**
- **"Take any amount** up to full capacity"
- **"Can cut the item to fit"**

- **"Maximize value with fractional parts"**
- Implies **continuous selection**

🧠 Think: Take as much as you want, even partially

🆚 Summary Table

Feature	0/1 Knapsack	Fractional Knapsack
Items divisible?	❌ No (only whole items)	✅ Yes (can take part of an item)
Approach	❌ Greedy fails → use **Dynamic Programming**	✅ Greedy approach works
Solution type	Integer / Subset	Real-valued / Continuous
Common use case	Discrete items like laptops, books	Liquid or divisible goods like gold, fuel
Optimization strategy	DP (with table)	Sort by profit/weight ratio, take greedily

---

✅ Quick Test:

Question: Can I take part of an item?

- **Yes ➜ Fractional Knapsack**
- **No ➜ 0/1 Knapsack**

Here's a clear **comparison between Greedy and Dynamic Programming (DP)** based on their core principles, advantages, and best use cases:

🧠 Core Idea

Aspect	Greedy	Dynamic Programming

Strategy	Make the **locally optimal choice** at each step	Solve **subproblems** and combine their solutions
Decision Process	**One-shot** decision, no going back	**Explores all possibilities**, stores intermediate results
Solution Type	Builds a solution **step-by-step**, one path	Explores **all subproblem combinations**

🔁 Reusability

Aspect	Greedy	Dynamic Programming
Reuses subproblem results?	No	Yes (using memoization or tabulation)
Backtracking allowed?	No	Yes (DP can revisit and compare multiple paths)

✅ Requirements

Property	Greedy	Dynamic Programming
Greedy Choice Property	Required	Not required
Optimal Substructure	Required	Required

🏃 Performance

Aspect	Greedy	Dynamic Programming
Time Complexity	Usually **O(n log n)** or **O(n)**	Usually **O(n²)** or more
Memory Usage	Low	High (due to table/memo storage)

🎯 Accuracy

Aspect	Greedy	Dynamic Programming

Guarantees optimal result?	Only for specific problems	Yes, always (if implemented correctly)
Risk of wrong result	High if problem is not suitable	None (just slower sometimes)

🛠️ Example Problems

Problem	Best Approach
Fractional Knapsack	✅ Greedy
0/1 Knapsack	❌ Greedy fails → ✅ DP
Huffman Encoding	✅ Greedy
Activity Selection	✅ Greedy
Longest Common Subsequence	✅ DP
Matrix Chain Multiplication	✅ DP

📊 Summary Table

Feature	Greedy	Dynamic Programming
Local Decision Making	✅ Yes	❌ No
Reuses Subproblems	❌ No	✅ Yes
Always Optimal	❌ No (depends on problem)	✅ Yes
Faster Execution	✅ Usually	❌ Usually Slower
Requires Optimal Substructure	✅ Yes	✅ Yes
Requires Greedy Choice	✅ Yes	❌ No

🧠 What is Dynamic Programming?

Dynamic Programming is a method for solving complex problems by **breaking them down into simpler subproblems**, solving each subproblem **only once**, and **storing the result** for future use (called **memoization** or **tabulation**).

It is especially useful when the problem has **overlapping subproblems** and **optimal substructure**.

📌 Key Concepts of DP

✅ 1. Optimal Substructure

If the solution to a problem can be **constructed from solutions of its subproblems**, it has optimal substructure.

> Example: In Fibonacci, `Fib(n) = Fib(n-1) + Fib(n-2)`

✅ 2. Overlapping Subproblems

If a problem **reuses the same subproblems**, instead of solving them repeatedly, we **store** their results.

> Example: Calculating `Fib(5)` calls `Fib(4)` and `Fib(3)` which both call `Fib(2)` again — overlapping!

✅ 3. Memoization (Top-Down)

- Use **recursion** + **cache (dictionary/array)** to store already computed results.

python
CopyEdit

```
def fib(n, memo={}):
    if n <= 1:
        return n
    if n not in memo:
        memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

✅ 4. Tabulation (Bottom-Up)

- Solve the problem **iteratively**, starting from base cases, and build up to the final solution using a **table**.

python
CopyEdit

```
def fib(n):
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]
```

🛠️ Steps to Solve a DP Problem

1. **Identify if it's a DP problem**
    - Does it have **overlapping subproblems** and **optimal substructure**?
2. **Define the state**
    - What does `dp[i]` represent?
3. **Define the recurrence relation**
    - How can you build `dp[i]` using previous values?
4. **Decide between Top-Down (Memoization) or Bottom-Up (Tabulation)**
5. **Implement the base cases and build up**

💡 Classic DP Problems (for practice)

Problem	State Example

Fibonacci	`dp[i] = dp[i-1] + dp[i-2]`
0/1 Knapsack	`dp[i][w] = max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])`
Longest Common Subsequence (LCS)	`dp[i][j] = ...` based on `s1[i]` and `s2[j]`

📊 Memoization vs Tabulation Summary

Feature	Memoization (Top-Down)	Tabulation (Bottom-Up)
Uses recursion	✅ Yes	❌ No
Uses table	✅ Yes	✅ Yes
Speed	Slower (due to recursion overhead)	Faster
Stack overflow	Possible	Not possible

Good question 👍
When you see a problem statement, certain **keywords/phrases** signal that it's actually an **LCS (Longest Common Subsequence)** problem (not substring, not edit distance, etc.).

🔑 Keywords & Phrases for LCS Problems

Look for words like:

1. **"Longest sequence"**
    - Example: *Find the longest sequence present in both strings.*
    - The word **sequence** (not substring) is a giveaway.
2. **"Not necessarily contiguous"**

    - Subsequence problems usually specify that the characters don't need to be consecutive.
3. **"Maintain relative order"**
    - Example: *The order of characters must remain the same in both strings.*
4. **"Common subsequence" / "common pattern"**
    - When asked to find something that is *common to two strings* in terms of order.
5. **Output is a length or the subsequence itself**
    - If the problem says: *Find the length of the longest subsequence* or *print the subsequence*.

🚩 What it's NOT

- If the problem says **contiguous**, then it's *Longest Common Substring*, not subsequence.
- If it mentions **minimum edits (insert, delete, replace)** → that's *Edit Distance*, but the LCS formula is closely related.
- If it's about finding common **supersequence** → that's *Shortest Common Supersequence (SCS)*, which also uses LCS.

✅ Quick trigger words to spot **LCS**:

- "subsequence"
- "not necessarily consecutive"
- "relative order preserved"
- "longest sequence common to both"

Do you want me to also give you a **list of problem variations** (like LCS of 3 strings, LCS vs substring, etc.) so you can identify them more easily in exams/contests?

Got it 👍 Let's break this into two parts: **Backtracking basics** and then **solving the 15-puzzle using BFS/DFS**.

🔹 Backtracking Algorithm (Basics)

- **Definition**: Backtracking is a **systematic trial-and-error** technique for solving problems by building solutions incrementally and abandoning (backtracking from) a path as soon as it is clear it cannot lead to a valid solution.

- **General Idea**:

    1. Choose an option.
    2. Recurse (explore further).
    3. If the current choice leads to a dead end → **backtrack** (undo the choice, try another).
- **When used**: Constraint satisfaction problems like **N-Queens, Sudoku, Maze solving, Hamiltonian cycle, Word search puzzles**.

Steps in pseudocode:

```
function backtrack(solution):
  if solution is complete:
    process solution
  else:
    for choice in possible_choices:
      make the choice
      if choice is valid:
        backtrack(solution with choice)
      undo the choice (backtrack)
```

-

🔹 15 Puzzle (Basics)

The **15 Puzzle** is a sliding puzzle with tiles numbered 1–15 and one empty space (4×4 board). Goal = arrange tiles in order (1,2,3,...,15, empty).

- **State**: Current arrangement of the board.
- **Initial state**: Given scrambled puzzle.
- **Goal state**: Solved puzzle.

- **Operators**: Slide a tile **up, down, left, right** into the empty space.
- **State space**: All possible arrangements reachable from the initial state.

🔹 Solving with BFS

- **Breadth-First Search** explores states **level by level**.
- **Why useful?** BFS guarantees the **shortest number of moves** to reach the goal.
- **Steps**:
    - Start with the initial board state in a queue.
    - While queue not empty:
        - Dequeue a state.
        - If it is the goal → return solution.
        - Else generate all possible moves (sliding tiles).
        - Enqueue new states if not visited.
- **Drawbacks**:
    - Huge state space (~10^13 states for 15-puzzle).
    - Memory heavy. BFS is impractical for larger puzzles but works for smaller ones (like 8-puzzle).

🔹 Solving with DFS

- **Depth-First Search** explores states by going deep along one path before backtracking.
- **Why useful?** Simple to implement, uses less memory than BFS.
- **Steps**:
    - Start from initial state, push into stack (or recursive DFS).
    - Pop a state, check if goal.
    - If not, expand possible moves and push them into stack.
    - Backtrack if no more moves.
- **Drawbacks**:
    - DFS does **not guarantee shortest solution**.
    - May go very deep into unpromising branches.

🔹 Where Backtracking Fits

For puzzles like **15-puzzle**, backtracking is essentially DFS with pruning:

- Keep track of visited states (to avoid cycles).
- Stop exploring a path if it cannot possibly lead to a solution (e.g., depth limit or repeated configurations).

👉 So, in short:

- **Backtracking** = DFS with undoing wrong choices.
- **15 Puzzle with BFS** = finds shortest solution but memory expensive.
- **15 Puzzle with DFS** = memory efficient but may not find shortest solution.

Would you like me to **show a small worked example** (say, a 3×3 **8-puzzle**) with BFS and DFS step expansions? That's easier to visualize than the 15-puzzle.

Great question 👍 You want to know **what kind of exam/assignment questions can come** for the **15 Puzzle** and the **N-Queens problem**, and also the **keywords to recognize them quickly**.

🔹 1. 15 Puzzle

✅ Types of Questions

1. **Conceptual / Theory**
    - Explain the state space of the 15 puzzle.
    - Why is BFS guaranteed to give the shortest solution?
    - Compare BFS vs DFS for solving the 15 puzzle.
    - What is the time complexity of solving the 15 puzzle?
    - Why are some 15-puzzle configurations unsolvable?
2. **Algorithmic / Practical**
    - Given an initial configuration, show the first few steps of BFS/DFS.

    - Check if a given configuration is solvable (by counting inversions).
    - Write pseudocode to solve the puzzle using BFS.
    - Design a heuristic for solving the 15 puzzle (like Manhattan distance).
    - Implement the A* algorithm for the 15 puzzle.
3. **Application-based**

    - Why is the 15 puzzle used in AI search problems?
    - What are the real-life applications of solving sliding puzzles?

✅ Keywords to Spot

- "Sliding puzzle"
- "15 puzzle" / "8 puzzle"
- "Solvable / unsolvable state"
- "State space"
- "BFS / DFS / A* search"
- "Inversions"
- "Manhattan distance heuristic"
- "Optimal solution / shortest path"

🔷 2. N-Queens Problem

✅ Types of Questions

1. **Conceptual / Theory**

    - What is the backtracking algorithm?
    - Why can't two queens be in the same row/column/diagonal?
    - What is the time complexity of solving N-Queens using backtracking?
2. **Algorithmic / Practical**

    - Solve the 4-Queens problem step by step.
    - Write pseudocode for N-Queens using backtracking.
    - Show one valid solution for N=4 or N=8.
    - Modify the algorithm to count all solutions.
3. **Application-based**

    - Where is N-Queens used in real life (e.g., VLSI design, parallel processing, scheduling)?
    - Compare backtracking with other methods (like branch-and-bound or heuristics).

✅ Keywords to Spot

- "N-Queens problem" / "4 Queens" / "8 Queens"
- "Backtracking"
- "Constraint satisfaction"
- "Row / column / diagonal"
- "Safe position"
- "Recursive algorithm"
- "State space tree"
- "All possible solutions / number of solutions"

✅ **Shortcut tip**:

- If you see **sliding tiles, empty space, BFS/DFS, heuristics, solvability** → it's a **15 puzzle question**.
- If you see **chessboards, queens, rows/columns/diagonals, backtracking, safe positions** → it's an **N-Queens question**.

Do you want me to also make a **table comparing 15 Puzzle vs N-Queens** (keywords, algorithm, type of problem)? That way you can memorize them side by side.